TMHyperPay — API Client Management Documentation

Audience: Backend developers, frontend integration team, QA
Base URL: `https://<your-host>/api/ApiClient`
Last Updated: 2026-03-19

Overview

Every system that wants to call the TMHyperPay payment APIs must first be registered as an **API Client**. When a client is registered, a unique **API Key** is generated. This key is passed with every payment request to prove the caller is authorized.

This section covers how to: - Register a new API client and get its key - List all registered clients - Regenerate a lost or compromised API key - Activate or deactivate a client

> [!IMPORTANT] All endpoints in this controller require a valid **JWT Bearer Token** in the `Authorization` header. You must log in first via the Auth endpoints to get a token before calling any of these.

Authentication

Every request must include:

`Authorization: Bearer <your_jwt_token>`

If the token is missing or expired, the server returns HTTP `401 Unauthorized` before your request even reaches the endpoint.

Standard Response Envelope

All responses — success or failure — follow the same wrapper shape:

```
{
  "success": true | false,
  "message": "Human-readable description",
  "data": { ... }   // present only on success
}
```

Field	Type	Description
`success`	`bool`	`true` if the operation succeeded, `false` on any error
`message`	`string`	Describes the outcome or the reason for failure
`data`	`object`	The result payload. Only present when `success` is `true`

Endpoints

---

1. Register a New API Client

Why: Before any third-party system or service can use the HyperPay payment API, it must be registered. Registration creates a record in the database and generates a secure API key that the client will use to authenticate all future payment requests.

`POST /api/ApiClient/register`

Request Headers

Header	Value
`Authorization`	`Bearer <jwt_token>`
`Content-Type`	`application/json`

Request Body

```
{
  "clientName": "My Frontend App"
}
```

Field	Type	Required	Description
`clientName`	`string`	Yes	A human-readable name to identify this client (e.g. “Tamimi Mobile App”, “POS Terminal”). Max 100 characters.

Success Response `200 OK`

```
{
  "success": true,
  "message": "Client registered successfully. Store this API key securely.",
  "data": {
    "clientId": 3,
    "clientName": "My Frontend App",
    "apiKey": "aB3xK9mNqR7vYz2..."
  }
}
```

Field	Description
`clientId`	The unique integer ID assigned to this client. **Store this** — you’ll need it to regenerate keys, toggle status, etc.
`clientName`	The name you submitted
`apiKey`	The **raw, plain-text API key**. This is the **only time** it will ever be shown. It is not stored in plain text in the database — only a secure hash is kept. **Copy and store it immediately.**

> [!CAUTION] The `apiKey` in the registration response is shown **exactly once**. If you lose it, you cannot retrieve it — you must regenerate a new one. Treat it like a password.

Error Responses

Scenario	`success`	`message`
`clientName` is empty	`false`	`"Client name is required"`
Server error	`false`	`"Error registering client: <details>"`

---

2. List All API Clients

Why: Gives administrators a view of all registered clients — who they are, whether they are active, and when they last made a request. Useful for auditing and management UIs.

`GET /api/ApiClient/list`

Request Headers

Header	Value
`Authorization`	`Bearer <jwt_token>`

No request body needed.

Success Response `200 OK`

```
{
  "success": true,
  "message": "Clients retrieved successfully",
  "data": [
    {
      "id": 1,
      "clientName": "My Frontend App",
      "apiKey": "***...aB3xK9mN",
      "isActive": true,
      "createdAt": "2026-03-15T08:30:00Z",
      "lastUsedAt": "2026-03-19T10:45:00Z"
    },
    {
      "id": 2,
      "clientName": "POS Terminal",
      "apiKey": "***...zQ8wP1rJ",
      "isActive": false,
      "createdAt": "2026-02-10T12:00:00Z",
      "lastUsedAt": null
    }
  ]
}
```

Field	Description
`id`	Unique ID of the client
`clientName`	Name given at registration
`apiKey`	**Masked** — shows `***` followed by the last 8 characters of the stored hash. This is **not** the actual key; it's just for identification
`isActive`	`true` = client can make API calls. `false` = client is blocked
`createdAt`	UTC timestamp of when the client was registered
`lastUsedAt`	UTC timestamp of the last API call made by this client. `null` if never used

> [!NOTE] The `apiKey` field in this response is intentionally masked for security. It is derived from the stored hash, not the original key. It cannot be used to authenticate. It is shown only to help identify which key is in use.

---

3. Regenerate API Key

Why: If an API key is lost, forgotten, or potentially compromised (e.g. accidentally committed to a repo), you can invalidate the old key and issue a brand new one without having to delete and re-create the entire client record. The old key **stops working immediately** the moment regeneration succeeds.

`POST /api/ApiClient/regenerate-api-key`

Request Headers

Header	Value
`Authorization`	`Bearer <jwt_token>`
`Content-Type`	`application/json`

Request Body

```
{
  "clientId": 3
}
```

Field	Type	Required	Description
`clientId`	`int`	Yes	The ID of the client whose key you want to regenerate (obtained from the `register` or `list` responses)

Success Response `200 OK`

```
{
  "success": true,
  "message": "API key regenerated successfully. Store this new key securely.",
  "data": {
    "clientId": 3,
    "clientName": "My Frontend App",
    "newApiKey": "xR7vYz2aB3mNqKp9...",
    "regenerationCount": 2,
    "regenerationsRemaining": 3,
    "lastRegeneratedAt": "2026-03-19T10:13:47Z"
  }
}
```

Field	Description
`clientId`	ID of the client
`clientName`	Name of the client
`newApiKey`	The **new raw API key**. Shown **only once** — save it immediately
`regenerationCount`	How many times this client's key has been regenerated so far (including this one)
`regenerationsRemaining`	How many more regenerations are still allowed before hitting the limit
`lastRegeneratedAt`	UTC timestamp of this regeneration

> [!CAUTION] Just like registration, `newApiKey` is shown **only once**. If you lose it, you'll need to regenerate again (consuming another slot from your limit).

Regeneration Limit Each client has a **maximum regeneration limit** (default: **5**). This is a safety guard to prevent abuse — if someone keeps regenerating keys, it creates a natural audit trail and eventually gets blocked.

- When the limit is **reached**, the endpoint returns a failure response and the old key remains valid.
- To increase the limit, an administrator must update it directly in the database (or via an admin endpoint).

Error when limit is reached:

```
{
  "success": false,
  "message": "Maximum regeneration limit of 5 has been reached for this client. Contact an a
}
```

All Error Responses

Scenario	`success`	`message`
`clientId` not found	`false`	`"Client with ID X not found"`
Regeneration limit reached	`false`	`"Maximum regeneration limit of N has been reached..."`
Server error	`false`	`"Error regenerating API key: <details>"`

---

4. Toggle Client Status (Activate / Deactivate)

Why: Sometimes you need to temporarily block a client from making API calls — for example, if you suspect misuse, or if the client's subscription has expired — without permanently deleting their record. This toggle flips between active and inactive.

`POST /api/ApiClient/toggle`

Request Headers

Header	Value
`Authorization`	`Bearer <jwt_token>`
`Content-Type`	`application/json`

Request Body

```
{
  "clientId": 3
}
```

Field	Type	Required	Description
`clientId`	`int`	Yes	ID of the client to activate or deactivate

How it works This endpoint reads the **current** `isActive` value and flips it: - If currently `true` → sets to `false` (deactivates) - If currently `false` → sets to `true` (activates)

You do **not** pass the desired state — it always toggles.

Success Response `200 OK`

```
{
  "success": true,
  "message": "Client deactivated successfully",
  "data": {
    "clientId": 3,
    "isActive": false
  }
}
```

Field	Description
`clientId`	The ID of the affected client
`isActive`	The **new** status after the toggle

Error Responses

Scenario	`success`	`message`
`clientId` not found	`false`	`"Client not found"`
Server error	`false`	`"Error toggling client status: <details>"`

> [!NOTE] When a client is deactivated (`isActive: false`), any API calls made with their key will be rejected by the authentication middleware — they do not need to be deleted, just toggled off.

Database Model Reference

The `ApiClient` table stores the following fields:

Column	Type	Default	Description
`Id`	`int`	Auto-increment	Primary key
`ClientName`	`string`	—	Display name of the client
`ApiKeyHash`	`string`	—	Bcrypt/HMAC hash of the API key. The actual key is **never stored**
`ApiKeySalt`	`string`	—	Salt used during hashing
`IsActive`	`bool`	`true`	Whether this client is allowed to make API calls
`CreatedAt`	`DateTime`	`NOW()`	When the client was registered
`LastUsedAt`	`DateTime?`	`null`	When the client last made a successful API call
`UserId`	`int?`	`null`	ID of the admin user who registered this client
`MaxRegenerations`	`int`	`5`	Maximum number of times the API key can be regenerated
`RegenerationCount`	`int`	`0`	Running counter of how many regenerations have occurred
`LastRegeneratedAt`	`DateTime?`	`null`	UTC timestamp of the most recent key regeneration
`LastRegeneratedByIp`	`string?`	`null`	IP address that triggered the most recent regeneration

Audit Log Events

Every important action is written to the audit log automatically. Here are the event codes you will see:

Event Code	Trigger
`CLIENT_REGISTERED`	A new client was successfully registered
`CLIENT_REGISTRATION_FAILED`	Registration was attempted but failed (e.g. missing name)
`CLIENT_REGISTRATION_ERROR`	Unexpected server error during registration
`API_KEY_REGENERATED`	API key was successfully regenerated
`API_KEY_REGEN_FAILED`	Regeneration attempted for a non-existent client
`API_KEY_REGEN_LIMIT_REACHED`	Regeneration blocked because `MaxRegenerations` was reached
`API_KEY_REGEN_ERROR`	Unexpected server error during regeneration
`CLIENT_STATUS_CHANGED`	Client was activated or deactivated
`CLIENT_TOGGLE_FAILED`	Toggle attempted for a non-existent client
`CLIENT_TOGGLE_ERROR`	Unexpected server error during toggle

Frontend Integration Guide

Typical Workflow

```
sequenceDiagram
    participant Admin as Admin UI
    participant API as TMHyperPay API
    participant DB as Database

    Admin->>API: POST /api/Auth/login
    API-->>Admin: JWT Token

    Admin->>API: POST /api/ApiClient/register { clientName }
    API->>DB: Create ApiClient row, store hash
    API-->>Admin: { clientId, apiKey } ← COPY THIS KEY NOW

    Admin->>API: GET /api/ApiClient/list
    API-->>Admin: All clients (masked keys)

    Admin->>API: POST /api/ApiClient/toggle { clientId }
```

```
API-->>Admin: { isActive: false }

Admin->>API: POST /api/ApiClient/regenerate-api-key { clientId }
API-->>Admin: { newApiKey } ← COPY THIS KEY NOW
```

Key Points for the Frontend Team

1. **Always store the API key immediately** after `register` or `regenerate-api-key`. Display it in a modal with a copy button and a warning. There is no "show again" button.
2. **`clientId` is your primary handle** for all subsequent operations. Store it alongside the client name in your frontend state after registration.
3. **The `apiKey` field in `list` is masked** — it's not usable for authentication. Don't confuse it with the real key.
4. **Toggle is stateless** — you don't tell it "activate" or "deactivate". It just flips. So always read `isActive` from the response to know the new state, and update your UI accordingly.
5. **Show `regenerationsRemaining`** in your admin UI so admins know how many regenerations are left before hitting the cap. Warn when it's at 1.
6. **All requests require a JWT token** — implement a request interceptor in your HTTP client to attach the `Authorization: Bearer <token>` header on every call.

Example: Register + Show Key (JavaScript/Fetch)

```javascript
async function registerClient(jwtToken, clientName) {
  const response = await fetch('/api/ApiClient/register', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${jwtToken}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ clientName })
  });

  const result = await response.json();

  if (result.success) {
    const { clientId, clientName, apiKey } = result.data;
    //  Show apiKey to the user NOW - it won't be available again
    showApiKeyModal(clientId, clientName, apiKey);
  } else {
    showError(result.message);
```

```
  }
}
```

Example: Regenerate Key (JavaScript/Fetch)

```
async function regenerateKey(jwtToken, clientId) {
  const response = await fetch('/api/ApiClient/regenerate-api-key', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${jwtToken}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ clientId })
  });

  const result = await response.json();

  if (result.success) {
    const { newApiKey, regenerationsRemaining } = result.data;
    //  Show newApiKey to the user NOW
    showApiKeyModal(clientId, null, newApiKey);
    updateRegenerationCounter(clientId, regenerationsRemaining);
  } else {
    showError(result.message); // Might be limit reached
  }
}
```